CHAPTER TWELVE
ADVANCED BEHAVIORAL MODELING

Events and Signals

- In the real world, things happen. Not only do things happen, but lots of things may happen at the same time, and at the most unexpected times. "Things that happen" are called events, and each one represents the specification of a significant occurrence that has a location in time and space.

- In the context of state machines, you use events to model the occurrence of a stimulus that can trigger a state transition. Events may include signals, calls, the passing of time, or a change in state.

- Events may be synchronous (occurring at the same time.) or asynchronous, so modeling events is wrapped up in the modeling of processes and threads.

- In the UML, each thing that happens is modeled as an event. An event is the specification of a significant occurrence that has a location in time and space. A signal, the passing of time, and a change of state are asynchronous events, representing events that can happen at arbitrary times.

- Calls are generally synchronous events, representing the invocation of an operation.

- The UML provides a graphical representation of an event, as Figure shows. This notation permits you to visualize the declaration of events (such as the signal OffHook), as well as the use of events to trigger a state transition

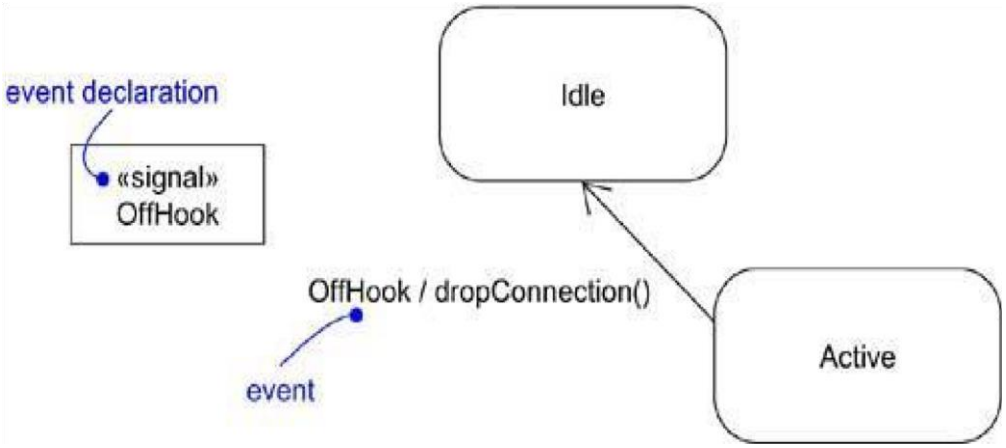


Fig: Events

Terms and Concepts

- An event is the specification of a significant occurrence that has a location in time and space.

In the context of state machines, an event is an occurrence of a stimulus that can trigger a state transition.

- A signal is a kind of event that represents the specification of an asynchronous stimulus communicated between instances.

Kinds of Events

- Events may be external or internal.
- External events are those that pass between the system and its actors. For example, the pushing of a button and an interrupt from a collision sensor are both examples of external events.
- Internal events are those that pass among the objects that live inside the system. An overflow exception is an example of an internal event.

- In the UML, you can model four kinds of events:
  - Signals
  - Calls
  - The passing of time, ▪ A change in state.

Signals

- A signal represents a named object that is dispatched (thrown) asynchronously by one object and then received (caught) by another.

- A signal may be sent as the action of a state transition in a state machine or the sending of a message in an interaction. The execution of an operation can also send signals.

- In the UML, as the below figure shows, you model signals (and exceptions) as stereotyped classes. You can use a dependency, stereotyped as send, to indicate that an operation sends a particular signal.

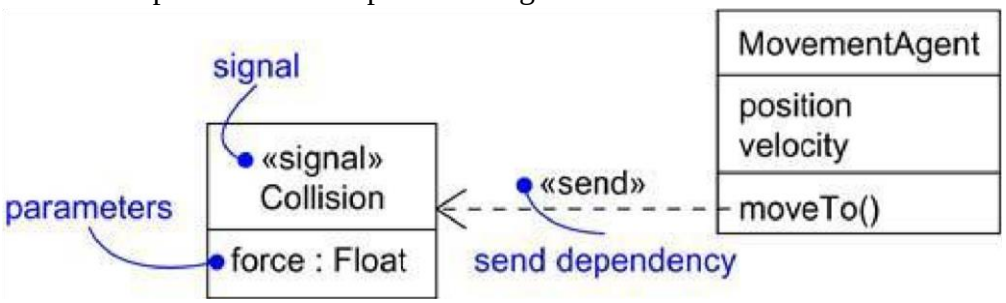


Fig: Signals

Call Events

- A signal event represents the occurrence of a signal, a call event represents the dispatch of an operation. In both cases, the event may trigger a state transition in a state machine.
- Whereas a signal is an asynchronous event, a call event is, in general, synchronous.
- This means that when an object invokes an operation on another object that has a state machine, control passes from the sender to the receiver, the transition is triggered by

-

the event, the operation is completed, the receiver transitions to a new state, and control returns to the sender.

As figure shows, modeling a call event is indistinguishable from modeling a signal event.

- In both cases, you show the event, along with its parameters, as the trigger for a state transition.

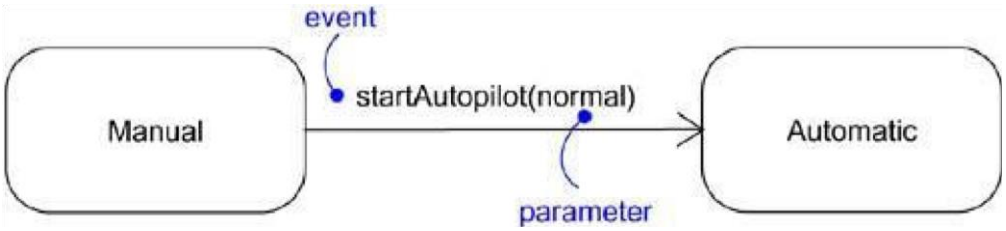


Fig: Call Events

Time and Change Events

- A time event is an event that represents the passage of time.
- As the below figure shows, in the UML you model a time event by using the keyword after followed by some expression that evaluates to a period of time.
- Such expressions can be simple (for example, after 2 seconds) or complex (for example, after 1 ms since exiting Idle).
- Unless you specify it explicitly, the starting time of such an expression is the time since entering the current state.

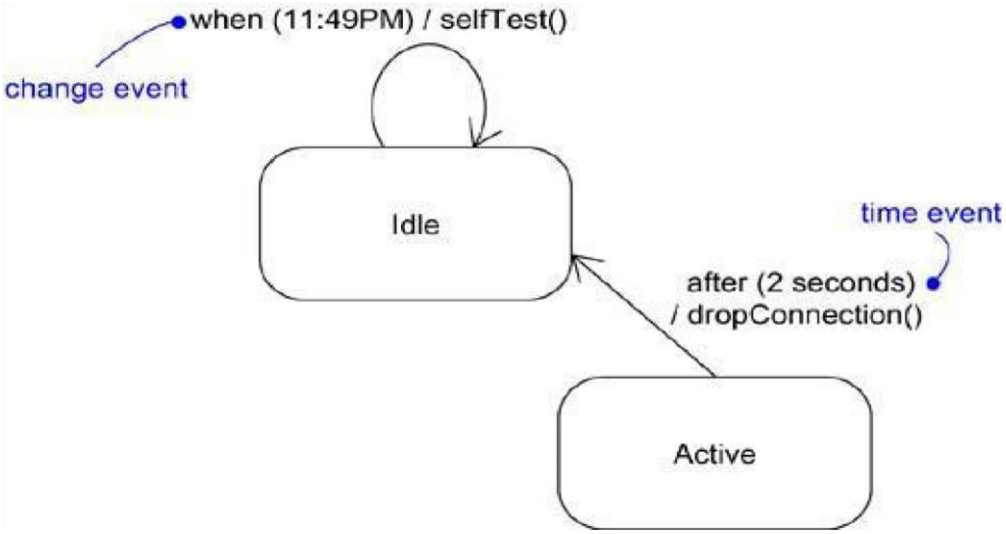


Fig: Time and Change Events

- A change event is an event that represents a change in state or the satisfaction of some condition.
- As the above figure shows, in the UML you model a change event by using the keyword when followed by some Boolean expression.

- You can use such expressions to mark an absolute time (such as when time = 11:59) or for the continuous test of an expression (for example, when altitude < 1000).

Sending and Receiving Events

- Signal events and call events involve at least two objects:
  The object that sends the signal or invokes the operation, and the object to which the event is directed.
- Because signals are asynchronous, and because asynchronous calls are themselves signals, the semantics of events interact with the semantics of active objects and passive objects.
- In the UML, you model the call events that an object may receive as operations on the class of the object.
- In the UML, you model the named signals that an object may receive by naming them in an extra compartment of the class, as shown in below figure.

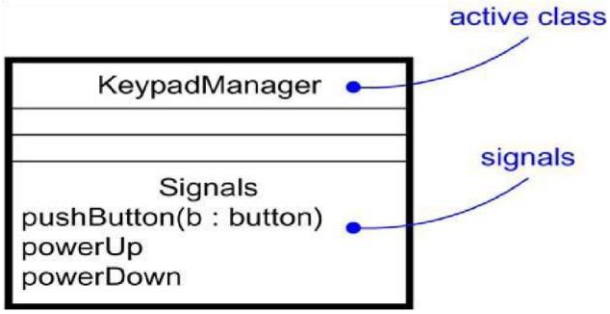


Fig: Signals and Active Classes

Common Modeling Techniques

Modeling a Family of Signals

To model a family of signals

- Consider all the different kinds of signals to which a given set of active objects may respond.
- Look for the common kinds of signals and place them in a generalization/specialization hierarchy using inheritance. Elevate more general ones and lower more specialized ones.
- Look for the opportunity for polymorphism in the state machines of these active objects.
- Where you find polymorphism, adjust the hierarchy as necessary by introducing intermediate abstract signals.

- The below figure models a family of signals that may be handled by an autonomous robot. Note that the root signal (RobotSignal) is abstract, which means that there may be no direct instances. This signal has two immediate concrete specializations (Collision and HardwareFault), one of which (HardwareFault) is further specialized. Note that the Collision signal has one parameter

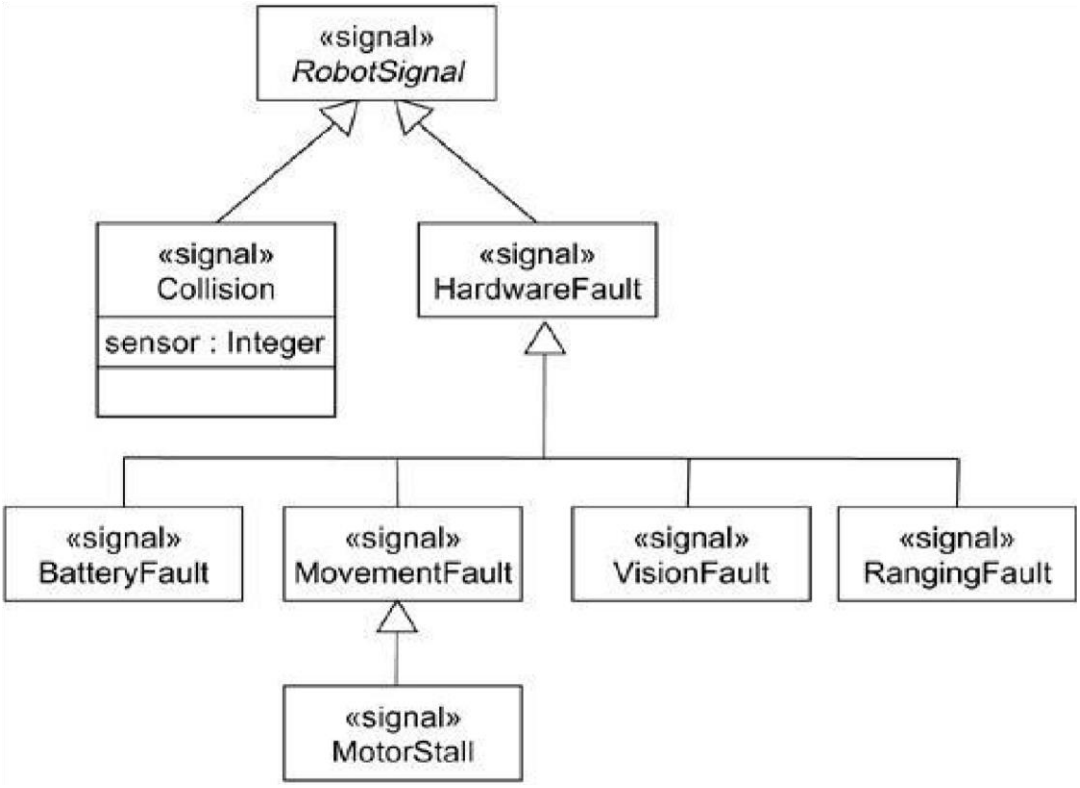


Fig: Modeling Families of Signals

Modeling Exceptions

- In the UML, exceptions are kinds of signals, which you model as stereotyped classes. Exceptions May be attached to specification operations.
- You model exceptions primarily to specify the kinds of exceptions that an object may throw through its operations.

To model exceptions

- For each class and interface, and for each operation of such elements, consider the exceptional conditions that may be raised.
- Arrange these exceptions in a hierarchy. Elevate general ones, lower specialized ones, and introduce intermediate exceptions, as necessary.
- For each operation, specify the exceptions that it may raise. You can do so explicitly (by showing send dependencies from an operation to its exceptions) or you can put this in the operation's specification.

The below figure models a hierarchy of exceptions that may be raised by a standard library of container classes, such as the template class Set. This hierarchy is headed by the abstract signal Exception and includes three specialized exceptions: Duplicate, Overflow, and Underflow. As shown, the add operation raises Duplicate and Overflow exceptions, and the remove operation raises only the Underflow exception. Alternatively, you could have

put these dependencies in the background by naming them in each operation's specification. Either way, by knowing which exceptions each operation may send, you can create clients that use the Set class correctly.

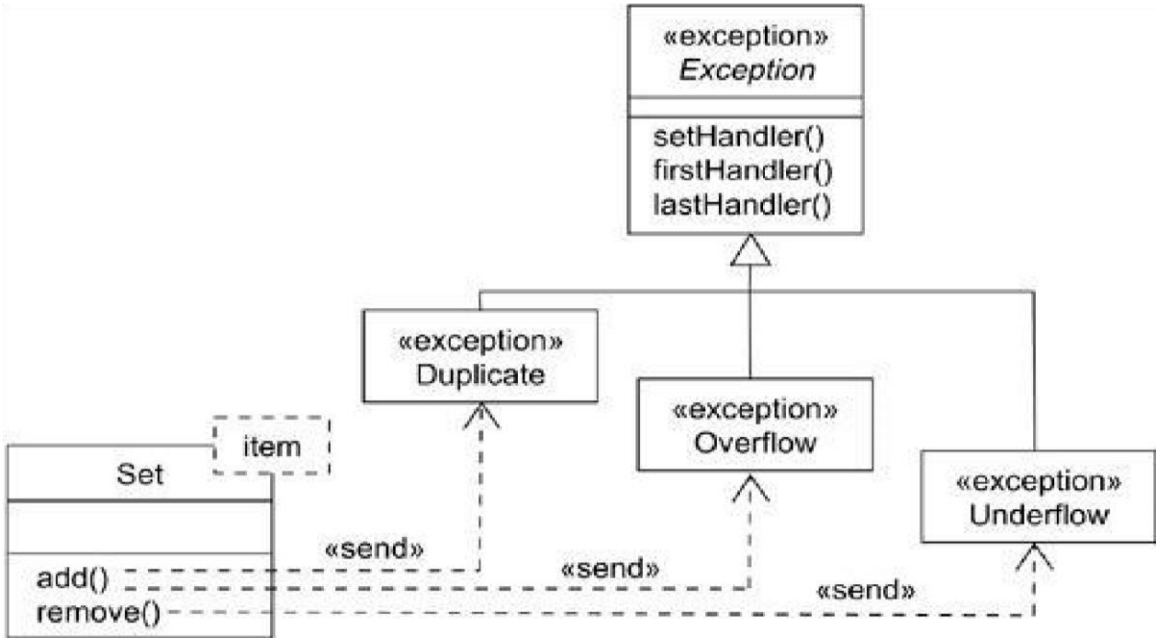


Fig: Modeling Exceptions

State Machines

- Using an interaction, you can model the behavior of a society of objects that work together.
- Using a state machine, you can model the behavior of an individual object.
- A state machine is a behavior that specifies the sequences of states an object goes through during its lifetime in response to events, together with its responses to those events.

- You use state machines to model the dynamic aspects of a system.

- You can visualize a state machine in two ways: by emphasizing the flow of control from activity to activity (using activity diagrams), or by emphasizing the potential states of the objects and the transitions among those states (using statechart diagrams).

- Well-structured state machines are like well-structured algorithms: They are efficient, simple, adaptable, and understandable.

- The UML provides a graphical representation of states, transitions, events, and actions, as Below figure shows. This notation permits you to visualize the behavior of an object in a way that lets you emphasize the important elements in the life of that object.

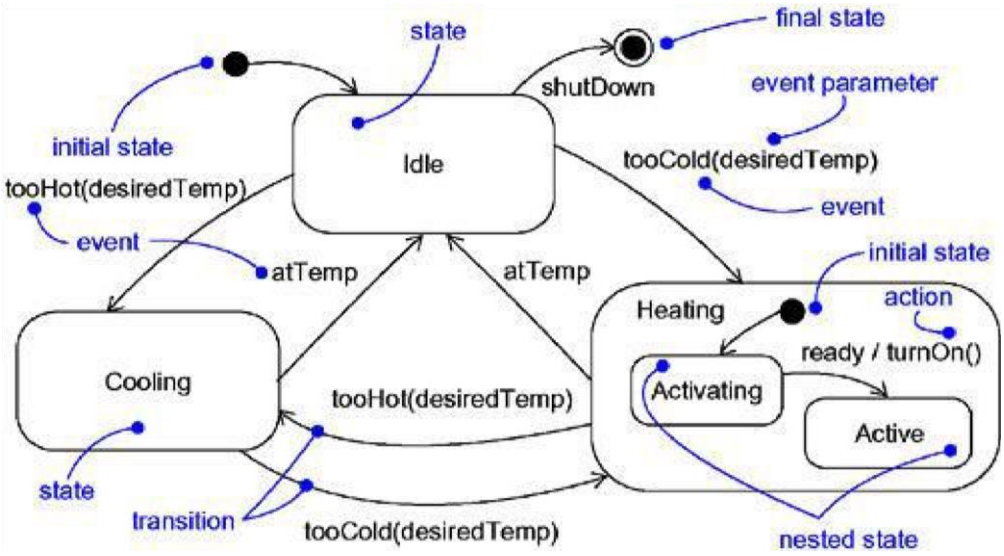


Fig: State Machines

Terms and Concepts

- A state machine is a behavior that specifies the sequences of states an object goes through during its lifetime in response to events, together with its responses to those events.

- A state is a condition or situation during the life of an object during which it satisfies some condition, performs some activity, or waits for some event.

- An event is the specification of a significant occurrence that has a location in time and space. In the context of state machines, an event is an occurrence of a stimulus that can trigger a state transition.

- A transition is a relationship between two states indicating that an object in the first state will perform certain actions and enter the second state when a specified event occurs and specified conditions are satisfied.

- An activity is ongoing nonatomic execution within a state machine.

- An Action is an executable atomic computation that results in a change in state of the model or the return of a value.

Astate has several parts.

1. Name	A textual string that distinguishes the state from other states; a state may be anonymous, meaning that it has no name
2. Entry/exit actions	Actions executed on entering and exiting the state, respectively
3. Internal transitions	Transitions that are handled without causing a change in state
4. Substates	The nested structure of a state, involving disjoint (sequentially active) or concurrent (concurrently active) substates
5. Deferred events	A list of events that are not handled in that state but, rather, are postponed and queued for handling by the object in another state

- Graphically, a state is rendered as a rectangle with rounded corners. A transition is rendered as a solid directed line.

States

- A state is a condition or situation during the life of an object during which it satisfies some condition, performs some activity, or waits for some event.

- An object remains in a state for a finite amount of time. For example, a Heater in a home might be in any of four states: Idle (waiting for a command to start heating the house), Activating (its gas is on, but it's waiting to come up to temperature), Active (its gas and blower are both on), and ShuttingDown (its gas is off but its blower is on, flushing residual heat from the system).

As below figure shows, you represent a state as a rectangle with rounded corners.

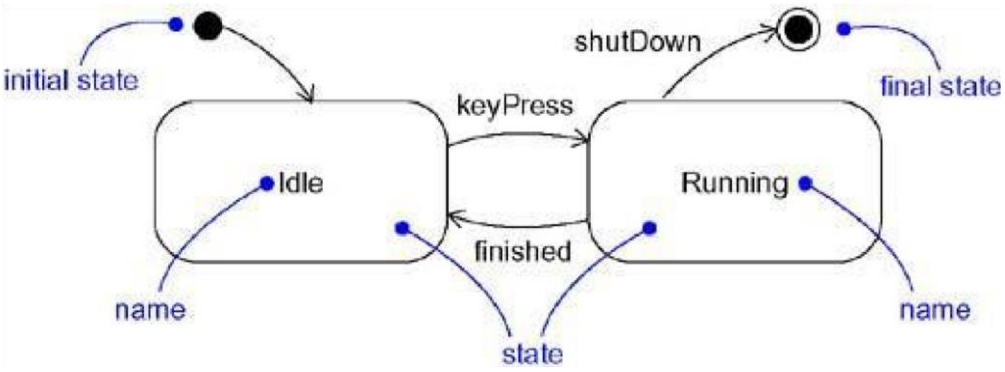


Fig: States

Initial and Final States

- As the above figure shows, there are two special states that may be defined for an object's state machine.

- First, there's the initial state, which indicates the default starting place for the state machine or substate. An initial state is represented as a filled black circle.

- Second, there's the final state, which indicates that the execution of the state machine or the enclosing state has been completed. A final state is represented as a filled black circle surrounded by an unfilled circle.

Transitions

- A transition is a relationship between two states indicating that an object in the first state will perform certain actions and enter the second state when a specified event occurs and specified conditions are satisfied.
- On such a change of state, the transition is said to fire. Until the transition fires, the object is said to be in the source state; after it fires, it is said to be in the target state.
- For example, a Heater might transition from the Idle to the Activating state when an event such as tooCold (with the parameter desiredTemp) occurs.

A transition has five parts.

1. Source state	The state affected by the transition; if an object is in the source state, an outgoing transition may fire when the object receives the trigger event of the transition and if the guard condition, if any, is satisfied
2. Event trigger	The event whose reception by the object in the source state makes the transition eligible to fire, providing its guard condition is satisfied
3. Guard condition	A Boolean expression that is evaluated when the transition is triggered by the reception of the event trigger; if the expression evaluates True, the transition is eligible to fire; if the expression evaluates False, the transition does not fire and if there is no other transition that could be triggered by that same event, the event is lost
4. Action	An executable atomic computation that may directly act on the object that owns the state machine, and indirectly on other objects that are visible to the object
5. Target state	The state that is active after the completion of the transition

- As below figure shows, a transition is rendered as a solid directed line from the source to the target state. A self-transition is a transition whose source and target states are the same.

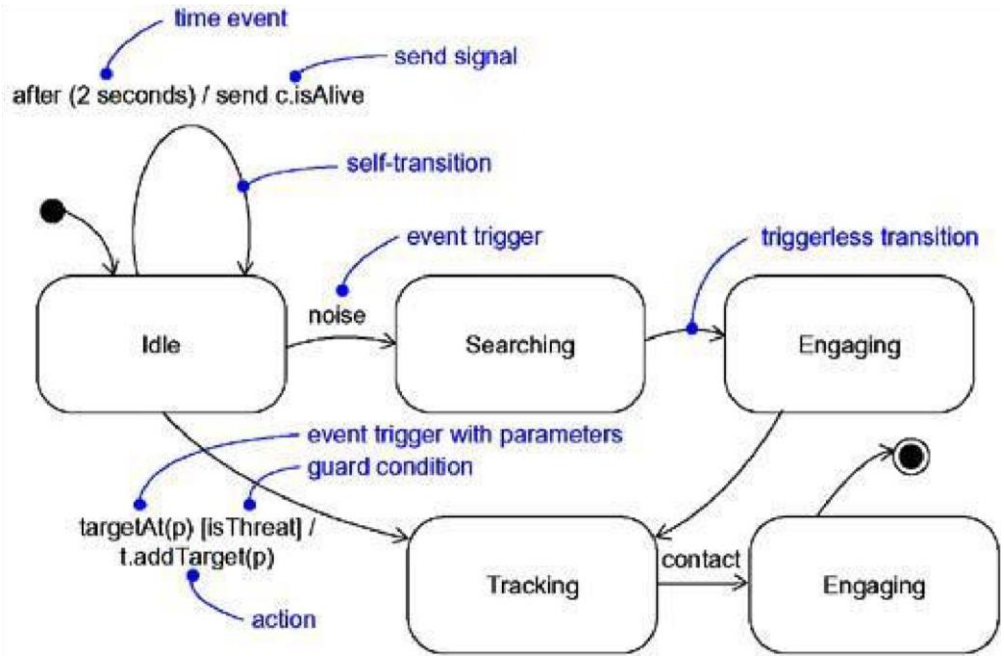


Fig: Transitions

Event Trigger

- An event is the specification of a significant occurrence that has a location in time and space.
- In the context of state machines, an event is an occurrence of a stimulus that can trigger a state transition.
- As shown in the previous figure, events may include signals, calls, the passing of time, or a change in state.
- A signal or a call may have parameters whose values are available to the transition, including expressions for the guard condition and action.
- It is also possible to have a triggerless transition, represented by a transition with no event trigger.
- A triggerless transition also called a completion transition• is triggered implicitly when its source state has completed its activity

Guard

- As the previous figure shows, a guard condition is rendered as a Boolean expression enclosed in square brackets and placed after the trigger event.
- A guard condition is evaluated only after the trigger event for its transition occurs.

Action

- An action is an executable atomic computation. Actions may include operation calls (to the object that owns the state machine, as well as to other visible objects), the creation or destruction of another object, or the sending of a signal to an object.

- As the previous figure shows, there's a special notation for sending a signal• the signal name is prefixed with the keyword send as a visual cue.

- An action is atomic, meaning that it cannot be interrupted by an event and therefore runs to completion. This is in contrast to an activity, which may be interrupted by other events.

Advanced States and Transitions

- The UML's state machines have a number of advanced features that help you to manage complex behavioral models

- Some of these advanced features include entry and exit actions, internal transitions, activities, and deferred events.

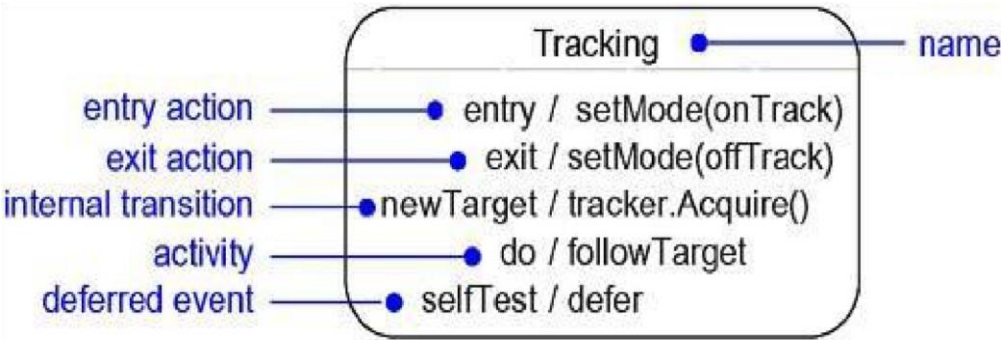


Fig A: Advanced States and Transitions

Entry and Exit Actions

- In a number of modeling situations, you'll want to dispatch the same action whenever you enter a state, no matter which transition led you there.

- Similarly, when you leave a state, you'll want to dispatch the same action no matter which transition led you away.

- As Figure A: shows, the UML provides a shorthand for this idiom. In the symbol for the state, you can include an entry action (marked by the keyword event entry) and an exit action (marked by the keyword event exit), together with an appropriate action. Whenever you enter the state, its entry action is dispatched; whenever you leave the state, its exit action is dispatched.

Internal Transitions

- Once inside a state, you'll encounter events you'll want to handle without leaving the state. These are called internal transitions, and they are subtly different from selftransitions

- As Figure A shows, the UML provides a shorthand for this idiom, as well (for example, for the event newTarget). In the symbol for the state, you can include an internal transition (marked by an event). Whenever you are in the state and that event is triggered, the corresponding action is dispatched without leaving and then reentering the state. Therefore, the event is handledwithout dispatching the state's exit and then entry actions.

Activities

- When an object is in a state, it generally sits idle, waiting for an event to occur. Sometimes, however, you may wish to model an ongoing activity. While in a state, the object does some work that will continue until it is interrupted by an event.

- As Figure A shows, in the UML, you use the special do transition to specify the work that's to be done inside a state after the entry action is dispatched. The activity of a do transition might name another state machine (such as followTarget). You can also specify a sequence of actions• for example, do / op1(a); op2(b); op3(c).

Deferred Events

- A deferred event is a list of events whose occurrence in the state is postponed until a state in which the listed events are not deferred becomes active, at which time they occur and may trigger transitions as if they had just occurred.

- As you can see in the previous figure, you can specify a deferred event by listing the event with the special action defer. In this example, selfTest events may happen while in the Tracking state, but they are held until the object is in the Engaging state, at which time it appears as if they just occurred.

Substates

- There’s one more feature of the UML's state machines substates that does even more to help you simplify the modeling of complex behaviors.
- A substate is a state that's nested inside another one.
- For example, a Heater might be in the Heating state, but also while in the Heating state, there might be a nested state called Activating. In this case, it's proper to say that the object is both Heating and Activating.
- A simple state is a state that has no substructure.
- A state that has substates that is, nested states is called a composite state.
- A composite state may contain either concurrent (orthogonal) or sequential (disjoint) substates.
- In the UML, you render a composite state just as you do a simple state, but with an optional graphic compartment that shows a nested state machine.
- Substates may be nested to any level.

Sequential Substates

- Using sequential substates, there's a simpler way to model this problem,
- The below figure B shows. Here, the Active state has a substructure, containing the substates Validating, Selecting, Processing, and Printing.
- The state of the ATM changes from Idle to Active when the customer enters a credit card in the machine.
- On entering the Active state, the entry action readCard is performed. Starting with the initial state of the substructure, control passes to the Validating state, then to the Selecting state, and then to the Processing state.
- After Processing, control may return to Selecting (if the customer has selected another transaction) or it may move on to Printing. After Printing, there's a triggerless transition back to the Idle state.
- Notice that the Active state has an exit action, which ejects the customer's credit card.

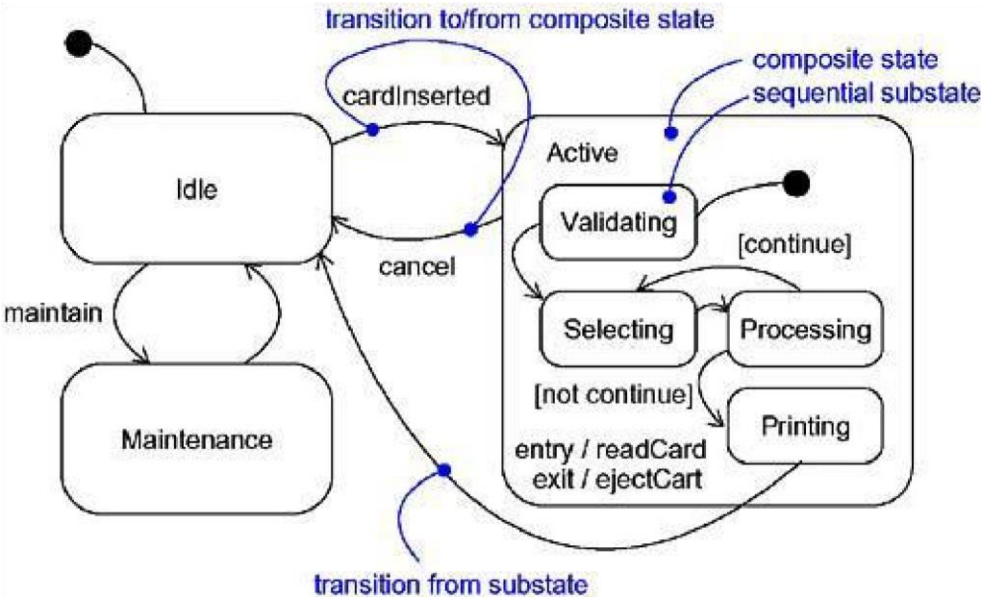


Fig B: Sequential Substates

History States

- In the UML, a simpler way to model this idiom is by using history states.
- A history state allows a composite state that contains sequential substates to remember the last substate that was active in it prior to the transition from the composite state.
- The below figure shows, you represent a shallow history state as a small circle containing the symbol H.

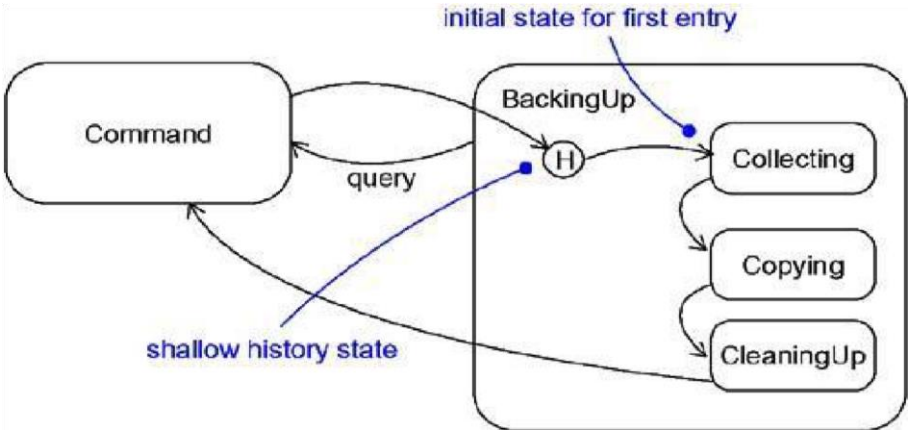


Fig: History State

Concurrent Substates

- Sequential substates are the most common kind of nested state machine.

- In certain modeling situations, however, you'll want to specify concurrent substates.
- These substates let you specify two or more state machines that execute in parallel in the context of the enclosing object.
- For example Maintenance state from below figure. Maintenance is decomposed into two concurrent substates,
- Testing and Commanding, shown by nesting them in the Maintenance state but separating them from one another with a dashed line.
- Each of these concurrent substates is further decomposed into sequential substates.
- When control passes from the Idle to the Maintenance state, control then forks to two concurrent flows the enclosing object will be in the Testing state and the Commanding state.
- Furthermore, while in the Commanding state, the enclosing object will be in the Waiting or the Command state.

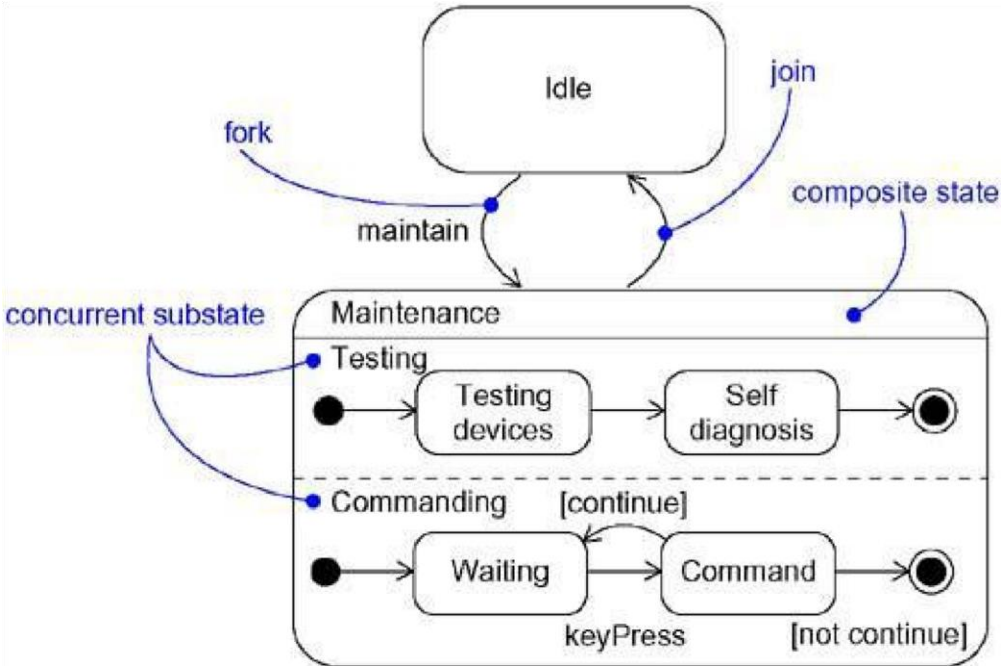


Fig: Concurrent Substates

Common Modeling Techniques

Modeling the Lifetime of an Object

- When you model the lifetime of an object, you essentially specify three things: the events to which the object can respond, the response to those events, and the impact of the past on current behavior.

- Modeling the lifetime of an object also involves deciding on the order in which the object can meaningfully respond to events, starting at the time of the object's creation and continuing until its destruction.

To model the lifetime of an object

- Set the context for the state machine, whether it is a class, a use case, or the system as a whole.

- If the context is a class or a use case, collect the neighboring classes, including any parents of the class and any classes reachable by associations or dependences. These neighbors are candidate targets for actions and are candidates for including in guard conditions.

- If the context is the system as a whole, narrow your focus to one behavior of the system. Theoretically, every object in the system may be a participant in a model of the system's lifetime, and except for the most trivial systems, a complete model would be intractable.
- Establish the initial and final states for the object. To guide the rest of your model, possibly state the pre- and post conditions of the initial and final states, respectively.

- Decide on the events to which this object may respond. If already specified, you'll find these in the object's interfaces; if not already specified, you'll have to consider which objects may interact with the object in your context, and then which events they may possibly dispatch.

- Starting from the initial state to the final state, lay out the top-level states the object may be in. Connect these states with transitions triggered by the appropriate events. Continue by adding actions to these transitions.

- Identify any entry or exit actions (especially if you find that the idiom they cover is used in the state machine).

- Expand these states as necessary by using substates.

- Check that all events mentioned in the state machine match events expected by the interface of the object. Similarly, check that all events expected by the interface of the object are handled by the state machine. Finally, look to places where you explicitly want to ignore events.

- Check that all actions mentioned in the state machine are sustained by the relationships, methods, and operations of the enclosing object.

- Trace through the state machine, either manually or by using tools, to check it against expected sequences of events and their responses. Be especially diligent in looking for unreachable states and states in which the machine may get stuck.

- After rearranging your state machine, check it against expected sequences again to ensure that you have not changed the object's semantics.

- For example, the below figure shows the state machine for the controller in a home security system, which is responsible for monitoring various sensors around the perimeter of the house.

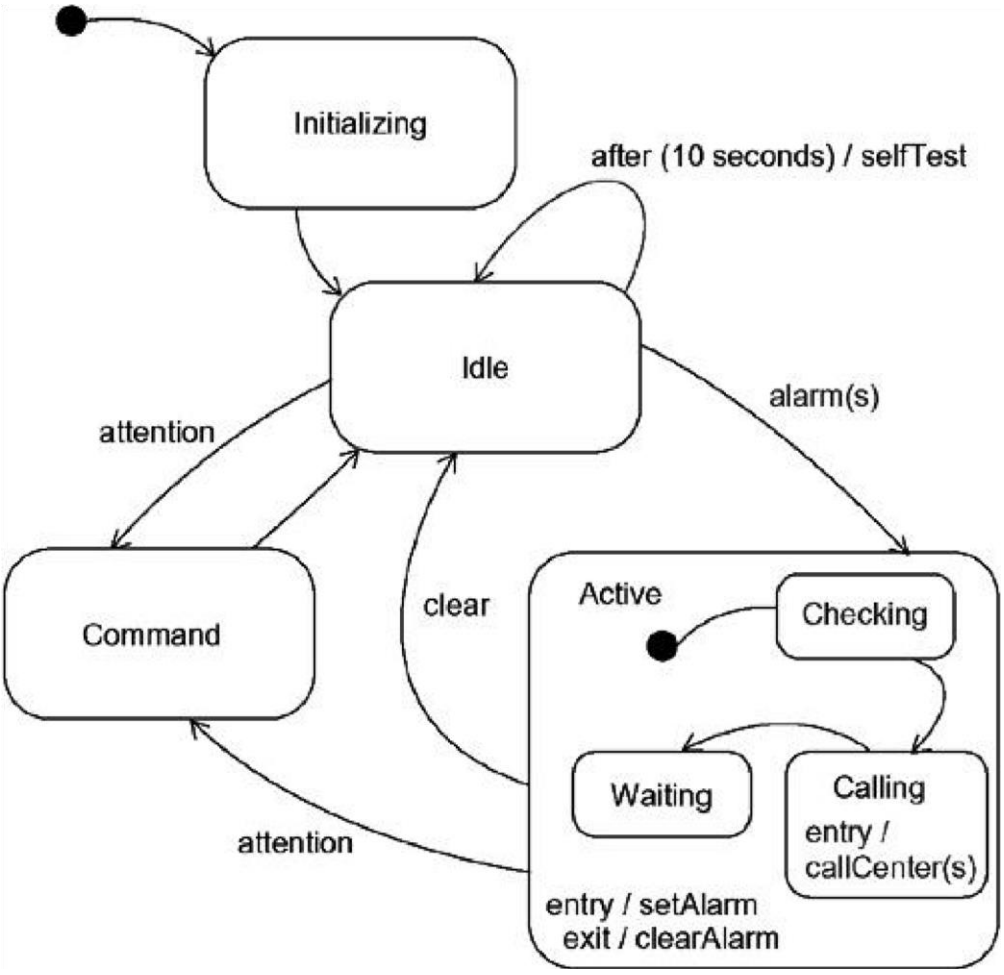


Fig: Modeling the Lifetime of an Object

Processes and Threads

- In the UML, you model each independent flow of control as an active object that represents a process or thread that can initiate control activity.

- A process is a heavyweight flow that can execute concurrently with other processes; a thread is a lightweight flow that can execute concurrently with other threads within the same process.

- In the UML, each independent flow of control is modeled as an active object. An active object is a process or thread that can initiate control activity. As for every kind of object, an active object is an instance of a class. In this case, an active object is an instance of an active class.

- The UML provides a graphical representation of an active class, the below figure shows.

- Active classes are kinds of classes, so have all the usual compartments for class name, attributes, and operations. Active classes often receive signals, which you typically enumerate in an extra compartment.

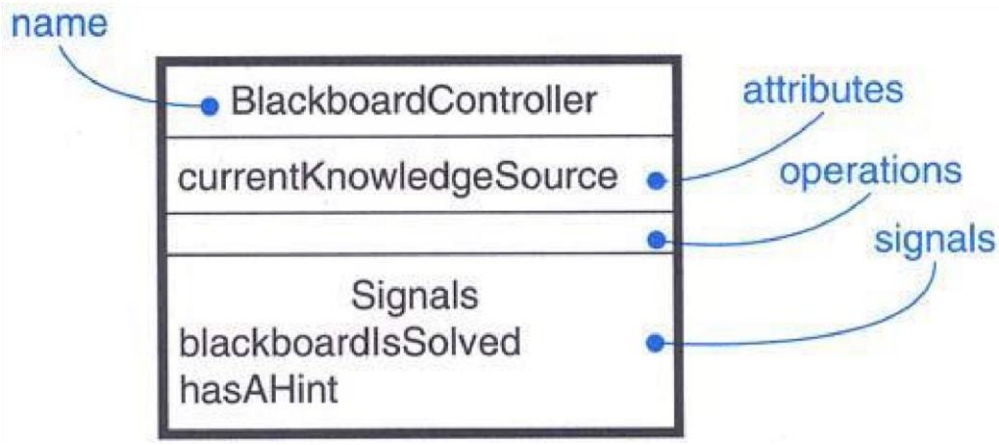


Fig: Active Class

Terms and Concepts

- An **active object** is an object that owns a process or thread and can initiate control activity. An
- **Active class** is a class whose instances are active objects.

- A **process** is a heavyweight flow that can execute concurrently with other processes.

- A **thread** is a lightweight flow that can execute concurrently with other threads within the same process.

- Graphically, an active class is rendered as a rectangle with thick lines. Processes and threads are rendered as stereotyped active classes (and also appear as sequences in interaction diagrams).

Classes and Events

- Active classes are just classes
- Active classes share the same properties as all other classes. Active classes may have instances.
- Active classes may have attributes and operations.
- Active classes may participate in dependency, generalization, and association (including aggregation) relationships.
- Active classes may use any of the UML's extensibility mechanisms, including stereotypes, tagged values, and constraints.
- Active classes may be the realization of interfaces.
- Active classes may be realized by collaborations, and the behavior of an active class may be specified by using state machines.

Standard Elements

The UML defines two standard stereotypes that apply to active classes.

1. process	Specifies a heavyweight flow that can execute concurrently with other processes
2. thread	Specifies a lightweight flow that can execute concurrently with other threads within the same process

- A process is heavyweight, which means that it is a thing known to the operating system itself and runs in an independent address space. Under most operating systems, such as Windows and Unix, each program runs as a process in its own address space
- A thread is lightweight. It may be known to the operating system itself. More often, it is hidden inside a heavier-weight process and runs inside the address space of the enclosing process. In Java, for example, a thread is a child of the class Thread.

Communication

- When objects collaborate with one another, they interact by passing messages from one to the other.

- In a system with both active and passive objects, there are four possible combinations of interaction that you must consider.

- First, a message may be passed from **one passive object to another**. Assuming there is only one flow of control passing through these objects at a time, such an interaction is nothing more than the simple invocation of an operation.

- Second, a message may be passed from **one active object to another**. When that happens, you have interprocess communication, and there are two possible styles of communication.
    - **First,** one active object might synchronously call an operation of another. That kind of communication has **rendezvous semantics**, which means that the caller calls the operation; the caller waits for the receiver to accept the call; the operation is invoked; a return object (if any) is passed back to the caller.

    - **Second,** one active object might asynchronously send a signal or call an operation of another object. That kind of communication has **mailbox semantics**, which means that the caller sends the signal or calls the operation and then continues on its independent way.
    - The two objects are not synchronized; rather, one object drops off a message for the other.

- In the UML, you render a synchronous message as a full arrow and an asynchronous message as a half arrow, as shown below figure.

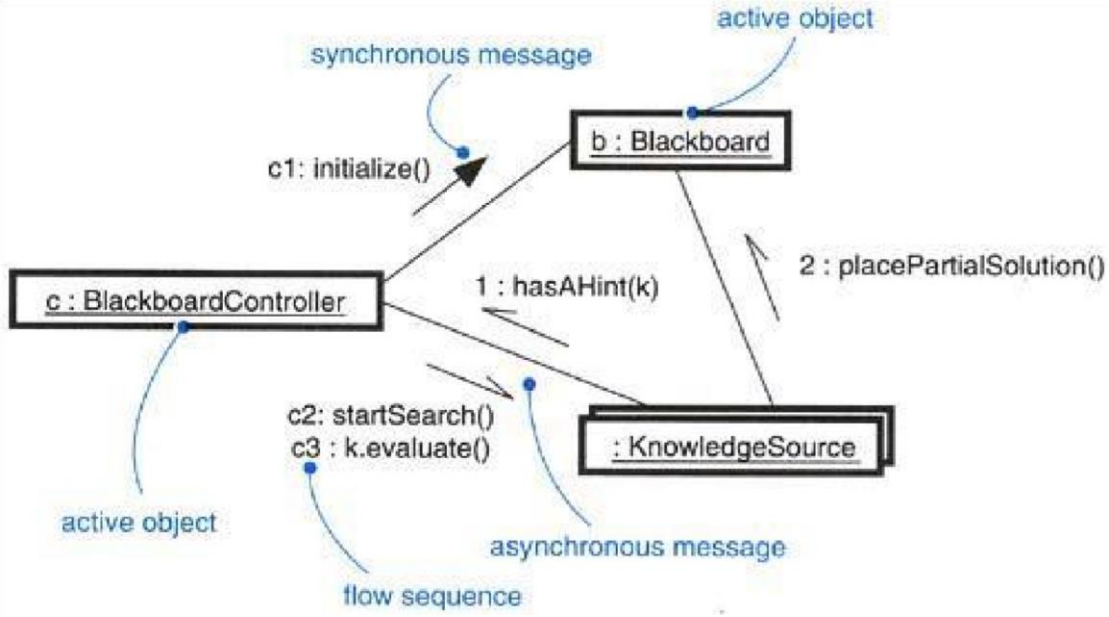


Fig: Communication

- Third, a message may be passed from **an active object to a passive object**. A difficulty arises if more than one active object at a time passes their flow of control through one passive object. In that situation, you have to model the synchronization of these two flows very carefully,

- Fourth, a message may be passed from **a passive object to an active one**. At first glance, this may seem illegal, but if you remember that every flow of control is rooted in some active object, you'll understand that a passive object passing a message to an active object has the same semantics as an active object passing a message to an active object.

Common Modeling Techniques

Modeling Multiple Flows of Control

To model multiple flows of control

- Identify the opportunities for concurrent action and reify each flow as an active class. Generalize common sets of active objects into an active class. Be careful not to over engineer the process view of your system by introducing too much concurrency.

- Consider a balanced distribution of responsibilities among these active classes, then examine the other active and passive classes with which each collaborates statically.

- Ensure that each active class is both tightly cohesive and loosely coupled relative to these neighboring classes and that each has the right set of attributes, operations, and signals.

- Capture these static decisions in class diagrams, explicitly highlighting each active class.

- Consider how each group of classes collaborates with one another dynamically. Capture those decisions in interaction diagrams. Explicitly show active objects as the root of such flows. Identify each related sequence by identifying it with the name of the active object.

- Pay close attention to communication among active objects. Apply synchronous and asynchronous messaging, as appropriate.

- Pay close attention to synchronization among these active objects and the passive objects with which they collaborate. Apply sequential, guarded, or concurrent operation semantics, as appropriate.

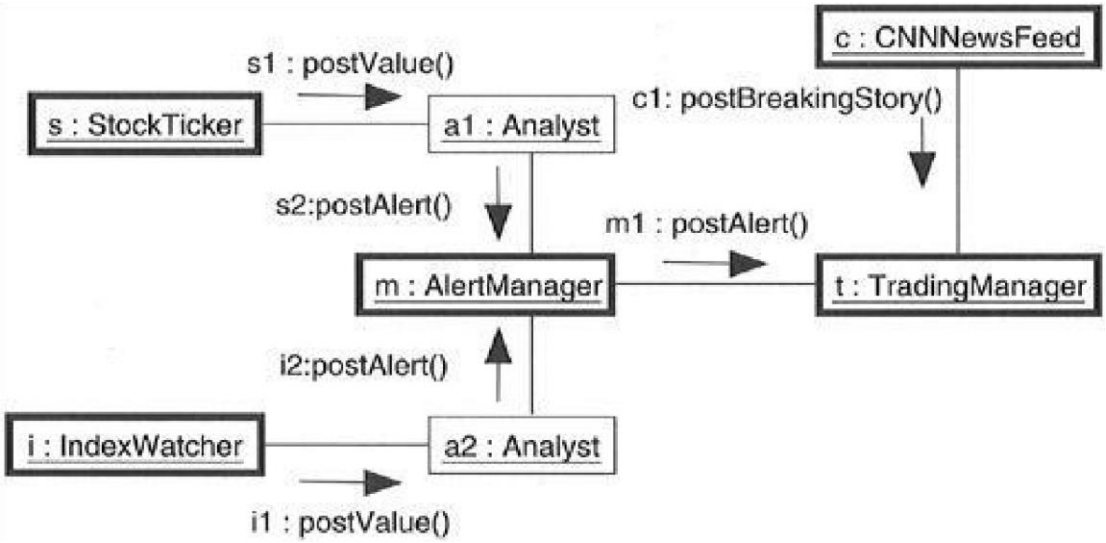


Fig: Modeling Flows of Control

Modeling Interprocess Communication

To model interprocess communication

- Consider which of these active objects represent processes and which represent threads. Distinguish them using the appropriate stereotype.

- Model messaging using asynchronous communication; model remote procedure calls using synchronous communication.

- Informally specify the underlying mechanism for communication by using notes, or more formally by using collaborations.

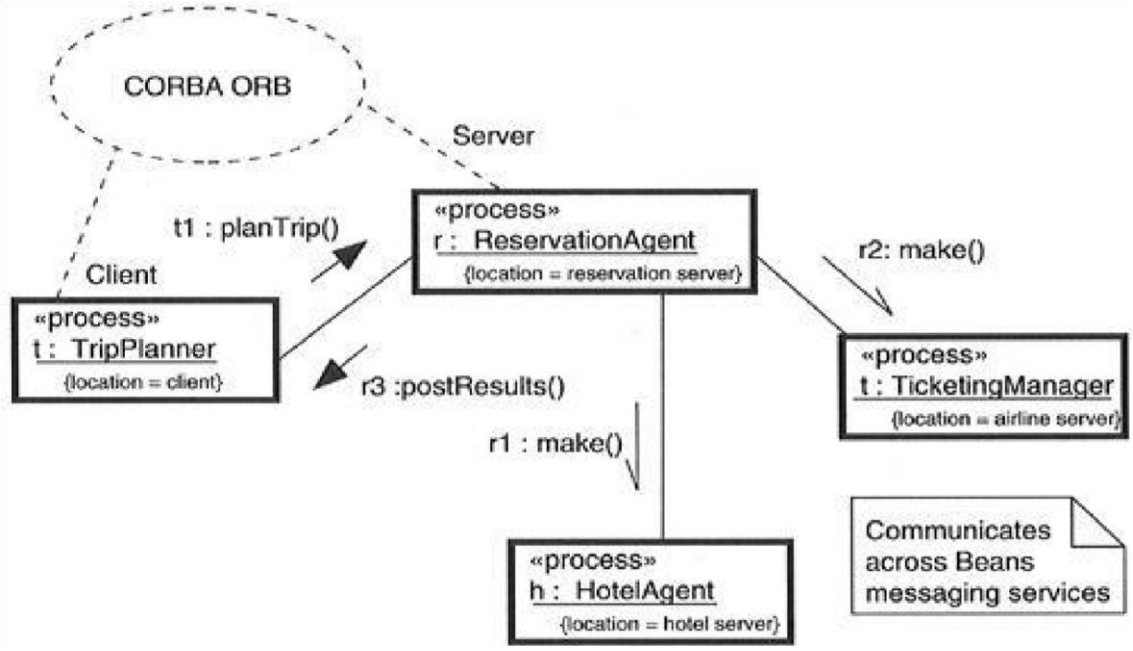


Fig: Modeling Interprocess Communication

Time and Space

- Modeling time and space is an essential element of any real time and/or distributed system.

- You use a number of the UML's features, including timing marks, time expressions, constraints, and tagged values, to visualize, specify, construct, and document these systems.

- To represent the modeling needs of real time and distributed systems, the UML provides a graphic representation for timing marks, time expressions, timing constraints, and location, as below figure shows.

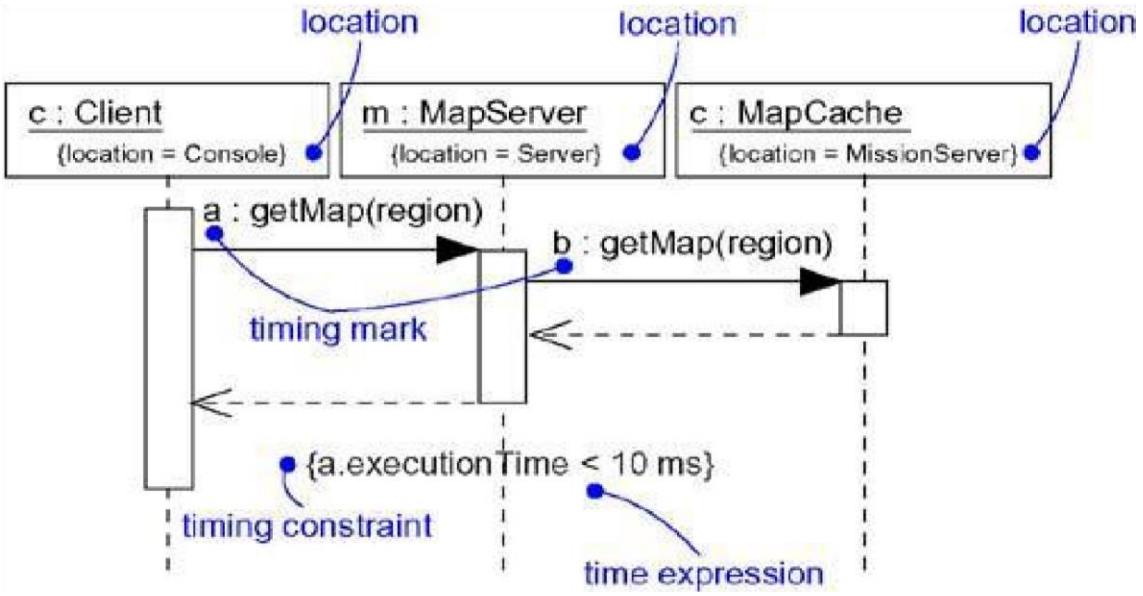


Fig: Timing Constraints and Location

Terms and Concepts

- A timing mark is a denotation for the time at which an event occurs.
- Graphically, a timing mark is formed as an expression from the name given to the message (which is typically different from the name of the action dispatched by the message).
- A time expression is an expression that evaluates to an absolute or relative value of time.
- A timing constraint is a semantic statement about the relative or absolute value of time.
- Graphically, a timing constraint is rendered as for any constraint• that is, a string enclosed by brackets and generally connected to an element by a dependency relationship.
- Location is the placement of a component on a node.
- Graphically, location is rendered as a tagged value• that is, a string enclosed by brackets and placed below an element's name, or as the nesting of components inside nodes.

Time

- Real time systems are, by their very name, time-critical systems. Events may happen at regular or irregular times; the response to an event must happen at predictable absolute times or at predictable times relative to the event itself.

- The passing of messages represents the dynamic aspect of any system, so when you model the time-critical nature of a system with the UML.

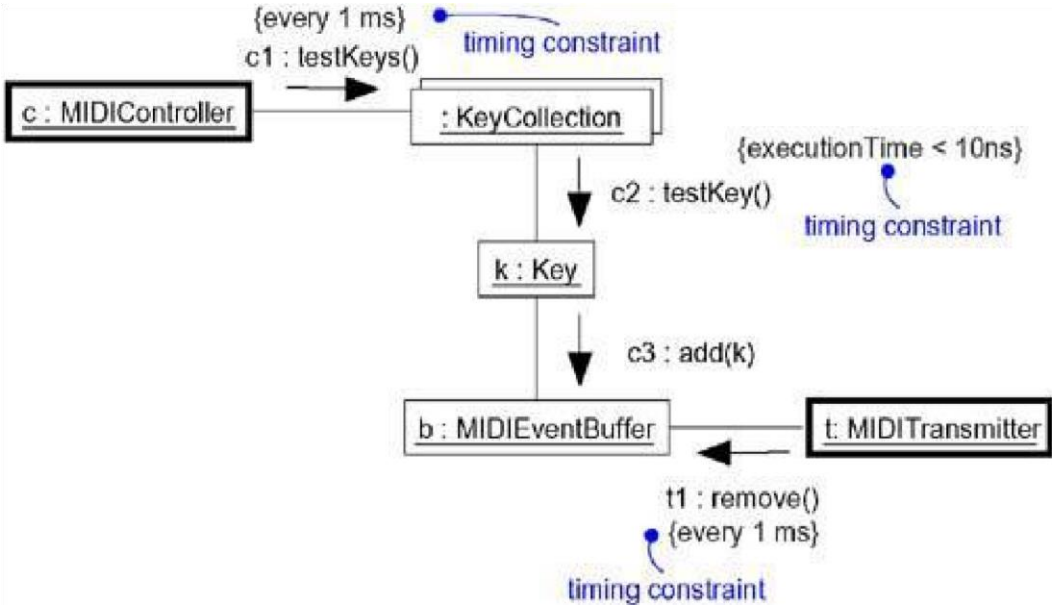


Fig: Time

Location

- In the UML, you model the deployment view of a system by using deployment diagrams that represent the topology of the processors and devices on which your system executes.

- Components such as executables, libraries, and tables reside on these nodes.

- Each instance of a node will own instances of certain components, and each instance of a component will be owned by exactly one instance of a node (although instances of the same kind of component may be spread across different nodes). For example, as Figure 23-3 shows, the executable component vision.exe may reside on the node named KioskServer.

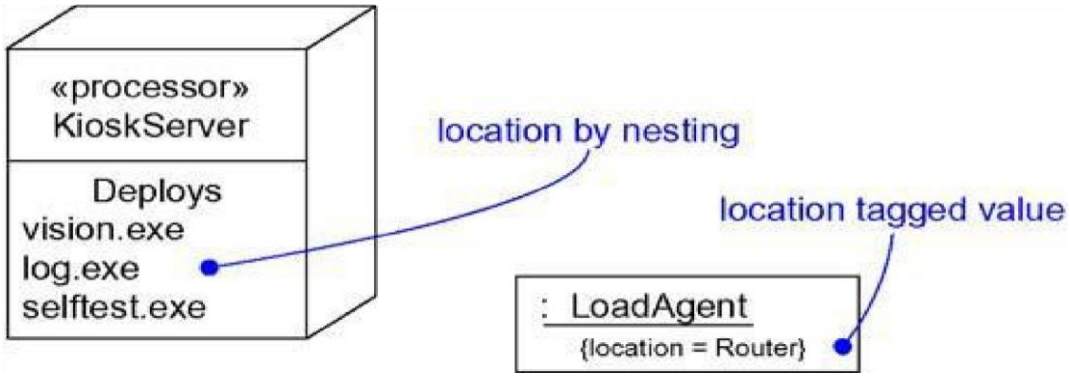


Fig: Location

Common Modeling Techniques

Modeling Timing Constraints

To model timing constraints

- For each event in an interaction, consider whether it must start at some absolute time. Model that real time property as a timing constraint on the message.

- For each interesting sequence of messages in an interaction, consider whether there is an associated maximum relative time for that sequence. Model that real time property as a timing constraint on the sequence.

- For each time critical operation in each class, consider its time complexity. Model those semantics as timing constraints on the operation.

- For example, as shown in figure, the left-most constraint specifies the repeating start time the call event refresh. Similarly, the center timing constraint specifies the maximum duration for calls to getImage. Finally, the right-most constraint specifies the time complexity of the call event getImage.

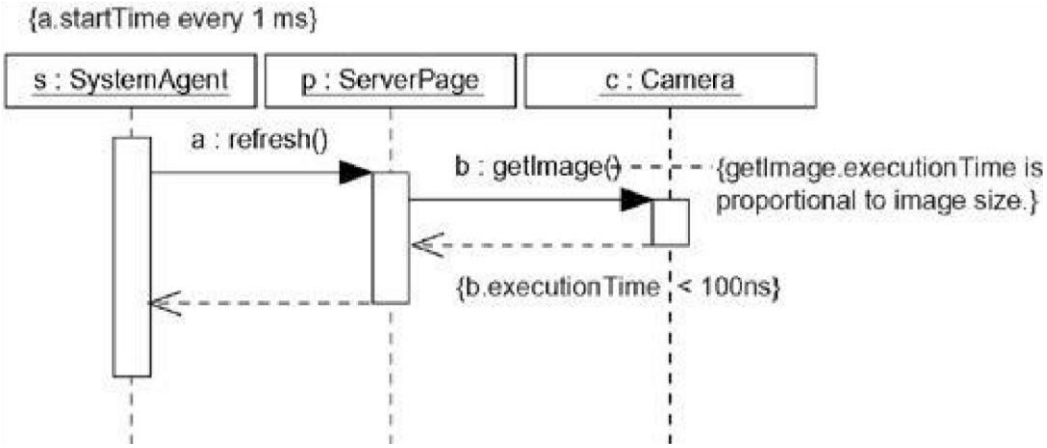


Fig: Modeling Timing Constraint

Modeling the Distribution of Objects

To model the distribution of objects

- For each interesting class of objects in your system, consider its locality of reference. In other words, consider all its neighbors and their locations.
- A tightly coupled locality will have neighboring objects close by; a loosely coupled one will have distant objects (and thus, there will be latency in communicating with them).
- Tentatively allocate objects closest to the actors that manipulate them.
- Next consider patterns of interaction among related sets of objects. Co-locate sets of objects that have high degrees of interaction, to reduce the cost of communication.
- Partition sets of objects that have low degrees of interaction.
- Next consider the distribution of responsibilities across the system. Redistribute your objects to balance the load of each node.
- Consider also issues of security, volatility, and quality of service, and redistribute your objects as appropriate.
- Render this allocation in one of two ways:
    1. By nesting objects in the nodes of a deployment diagram
    2. By explicitly indicating the location of the object as a tagged value
- Below figure provides an object diagram that models the distribution of certain objects in a retail system. The value of this diagram is that it lets you visualize the physical

distribution of certain key objects. As the diagram shows, two objects reside on a Workstation (the Order and Sales objects), two objects reside on a Server (the ObserverAgent and the Product objects), and one object resides on a DataWarehouse (the ProductTable object).

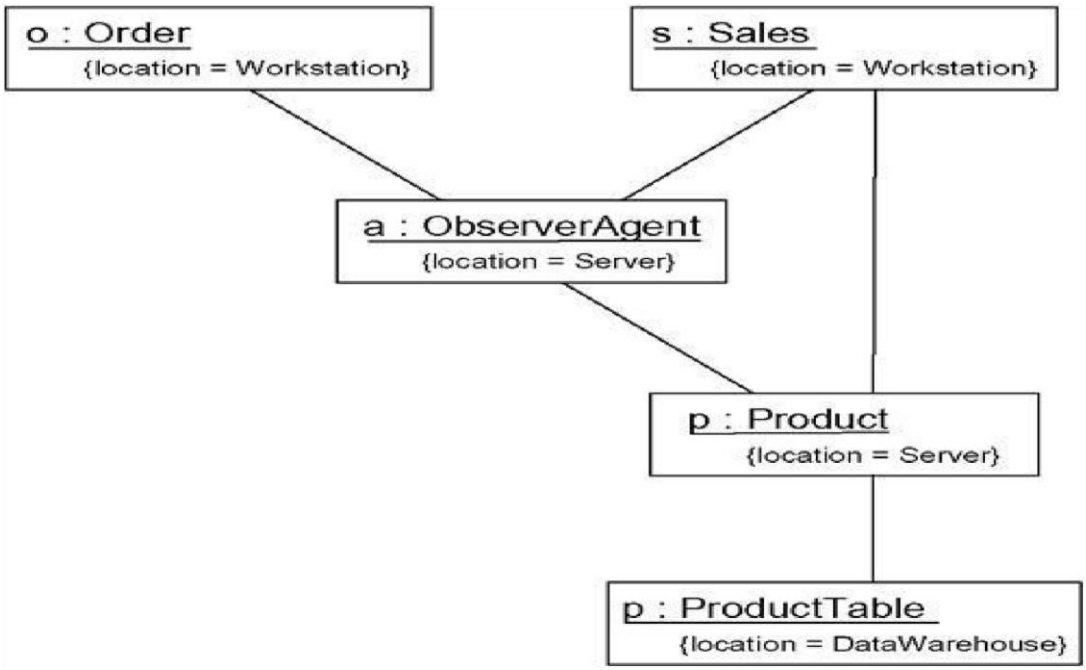


Fig: Modeling the Distribution of Objects

Modeling Objects that Migrate

To model the migration of objects

- Select an underlying mechanism for physically transporting objects across nodes.
- Render the allocation of an object to a node by explicitly indicating its location as a tagged value.
- Using the become and copy stereotyped messages, render the allocation of an object to a new node.
- Consider the issues of synchronization (keeping the state of cloned objects consistent) and identity (preserving the name of the object as it moves).
- The below figure provides a collaboration diagram that models the migration of a Web agent that moves from node to node, collecting information and bidding on resources in order to automatically deliver a lowest-cost travel ticket. Specifically, this diagram shows an instance (named t) of the class TravelAgent migrating from one server to another.

Along the way, the object interacts with anonymous Auctioneer instances at each node, eventually delivering a bid for the Itinerary object, located on the client server.

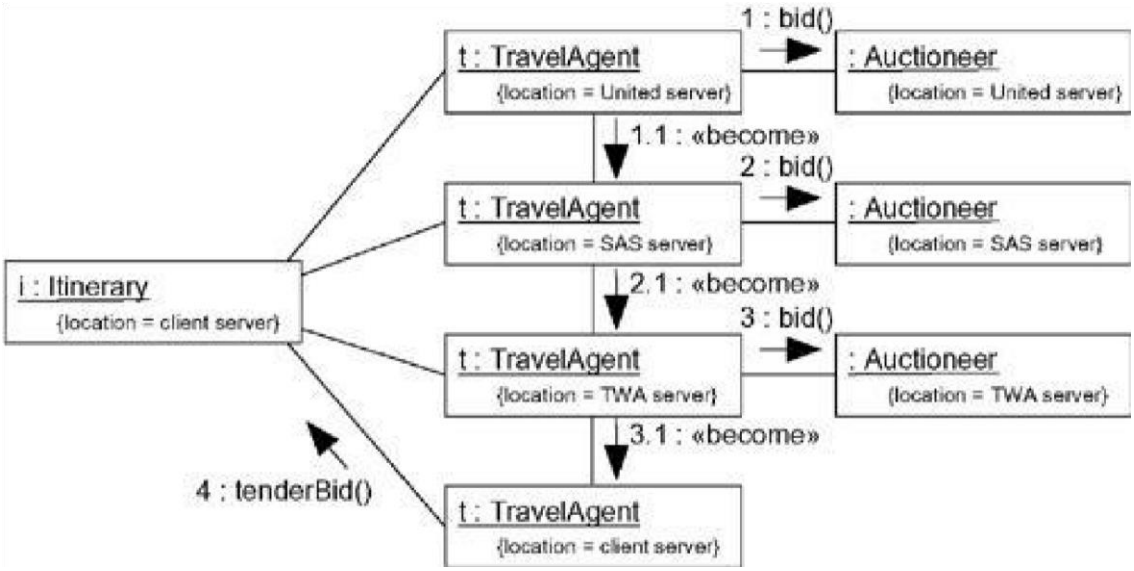


Fig: Modeling Objects that Migrate

State chart Diagrams

- Statechart diagrams are one of the five diagrams in the UML for modeling the dynamic aspects of systems.
- A statechart diagram shows a state machine.
- An activity diagram is a special case of a statechart diagram in which all or most of the states are activity states and all or most of the transitions are triggered by completion of activities in the source state.
- Both activity and statechart diagrams are useful in modeling the lifetime of an object. However, whereas an activity diagram shows flow of control from activity to activity, a statechart diagram shows flow of control from state to state.
- Statechart diagrams are not only important for modeling the dynamic aspects of a system, but also for constructing executable systems through forward and reverse engineering.
- In the UML, you model the event-ordered behavior of an object by using statechart diagrams.

- As below figure shows, a statechart diagram is simply a presentation of a state machine, emphasizing the flow of control from state to state.

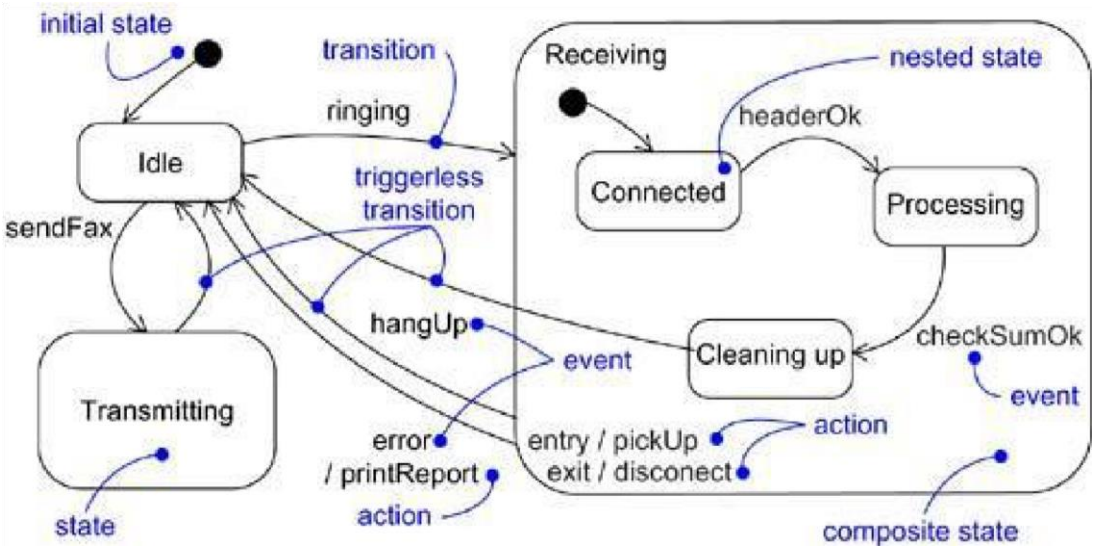


Fig: State chart Diagram

Terms and Concepts

- A statechart diagram shows a state machine, emphasizing the flow of control from state to state.

- A state machine is a behavior that specifies the sequences of states an object goes through during its lifetime in response to events, together with its responses to those events.

- A state is a condition or situation in the life of an object during which it satisfies some condition, performs some activity, or waits for some event.

- An event is the specification of a significant occurrence that has a location in time and space. In the context of state machines, an event is an occurrence of a stimulus that can trigger a state transition.

- A transition is a relationship between two states indicating that an object in the first state will perform certain actions and enter the second state when a specified event occurs and specified conditions are satisfied.

- An activity is ongoing nonatomic execution within a state machine.

- An action is an executable atomic computation that results in a change in state of the model or the return of a value.

- Graphically, a statechart diagram is a collection of vertices and arcs.

Common Properties

- A statechart diagram is just a special kind of diagram and shares the same common properties as do all other diagrams that is, a name and graphical contents that are a projection into a model.
- What distinguishes a statechart diagram from all other kinds of diagrams is its content.